Lab2: Computer Architectuur

Stijn Verhaeghe & Andres Bosman

February 2026

1 Introductie

In dit labo gaan we onderzoeken wat race conditions zijn en welke oplossingen er zijn. Het probleem is dat als twee threads tegelijkertijd dezelfde data proberen aan te passen, dit kan leiden tot een foutief eindresultaat. In figuur 4 zien we dat thread x T_q van het geheugen ophaalt en voert een bewerking op uit, nog voor dat het geheugen geupdate is met de nieuwe waarde. Ontstaat er een thread y die ook dezelfde waarde opvraagt uit het geheugen om een bewerking uit te voeren. thread x update het geheugen en daarna thread y dus we zijn de data van thread x verloren, dit is een groot probleem die tot fouten leidt.

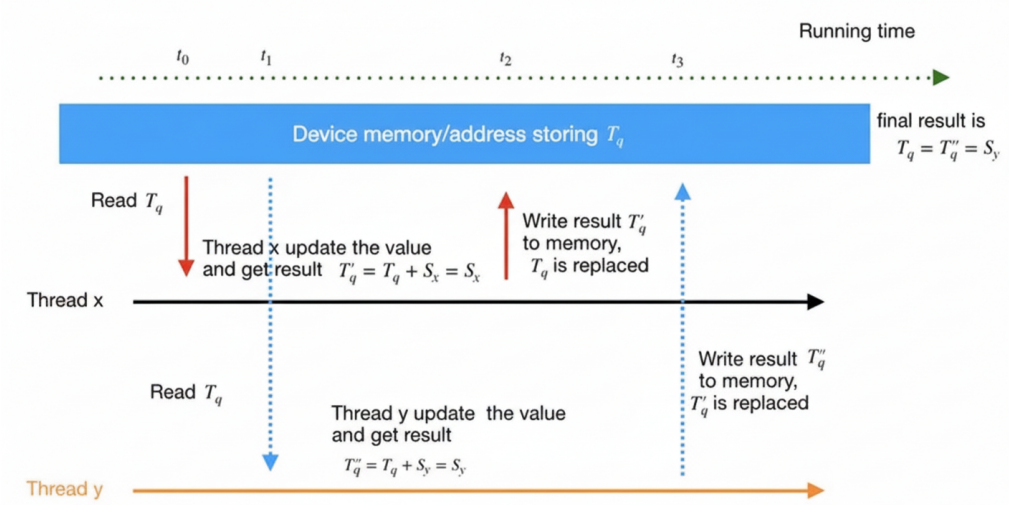


Figure 1: Voorbeeld race condition

De opdracht van dit labo is om de maximale waarde te detecteren in een array, we gaan dit op drie verschillende manieren doen:

1. Een nested for loop op de CPU,

2. Een GPU kernel die gebruik maakt van een atomic operatie, en

3. Een GPU kernel die gebruik maakt van reduction

2 Implementatie

2.1 nested for loop op de CPU

Voor de eerste manier van de maximale waarde te detecteren in een array gaan we dit implementeren op een CPU. Aan de hand van een for loop kunnen we dit gemakkelijk controleren telkens als er een grotere maximum tegen komen dan wordt die opgeslagen. Uit de simulaties (figuur 2) kunnen we besluiten dat de CPU in kleine arrays heel snel de grootste waarde kan vinden in grotere arrays vanaf een grootte van 100000, zien we dat de tijd dat de CPU nodig heeft sterk stijgt. De specifieke tijdsmeting per array grootte staat in de appendix 1.

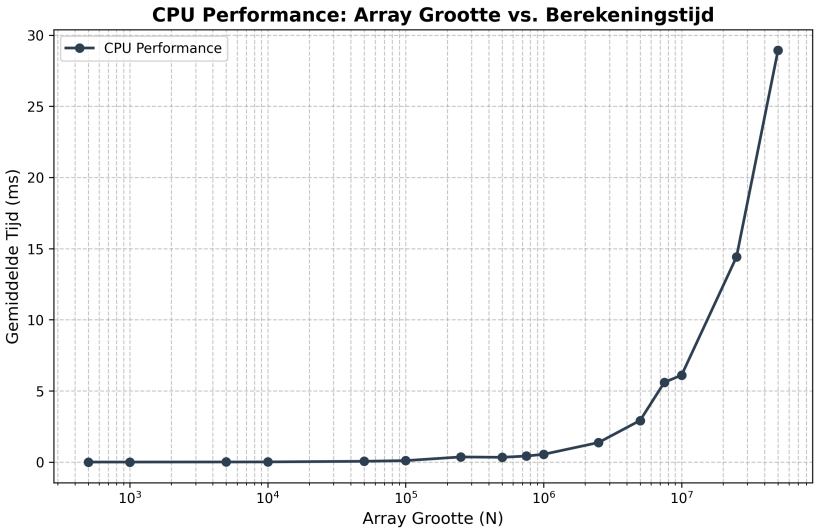


Figure 2: Prestatie CPU

2.2 Een GPU kernel die gebruik maakt van een atomic operatie

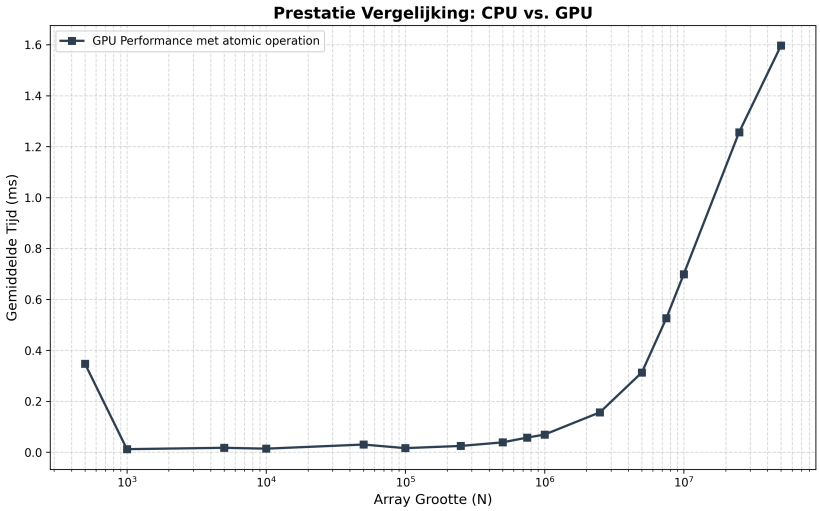


Figure 3: Prestatie van de GPU met de atomic operatie

Bij de atomic operatie merken we dat dit heel traag werkt voor kleine arrays (bijna 0,4ms), dan heel snel werkt tot de array bestaat uit ongeveer een miljoen elementen en dan sterker stijgt tot 10miljoen elementen. waarna het 1,6ms duurt vooraleer hij klaar is.

2.3 Een GPU kernel die gebruik maakt van reduction

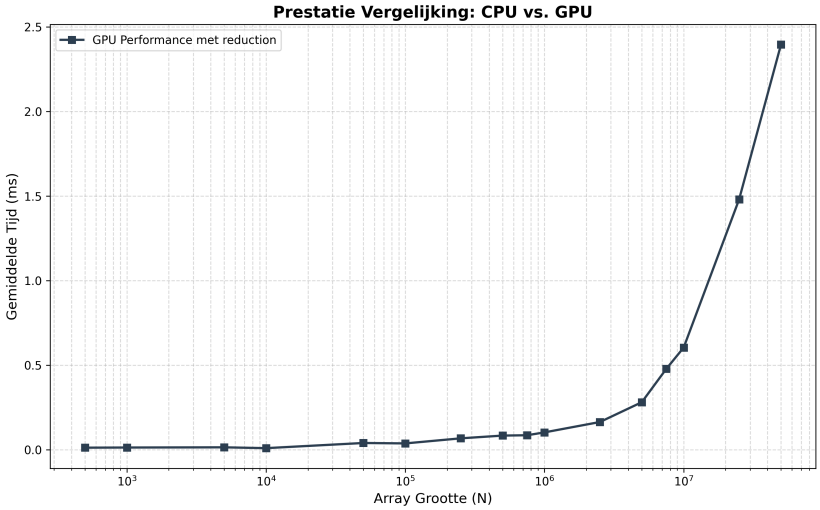


Figure 4: Prestatie van de GPU met de reduction methode

Reductie begint snel en neemt langzaam aan toe met 2,5ms voor 10miljoen elementen wat ongeveer 1,5 trager is dan de atomic operations. Het is dus beter om reductie te gebruiken bij arrays kleiner dan 1000 elementen en daarna is het beter om met atomic operations te werken.

3 Vragen en Analyse

Is het resultaat correct?

Om te controleren of de code correct werkt, hebben we een array met 10 elementen random gegenereert. Daarna hebben we de code laten runnen, waardoor we zeker weten dat de code correct werkt.

```
Inhoud array: 289383 930886 692777 636915 747793 238335 885386 760492 516649 641421
CPU | N:         10 | Max:  930886 | Gem. Tijd: 0.000088 ms (0.000000 s)
```

(a) CPU Output

```
Inhoud array: 734643 273795 629774 533585 650647 620397 815757 813001 668073 567837
N:         10 | Max:  815757 | Tijd: 0.0656 ms
```

(b) GPU Atomic Output

```
Inhoud array: 616681 893262 706927 816787 263094 293802 418821 681492 530248 533395
N:         10 | Max:  893262 | Tijd: 0.2203 ms
```

(c) GPU Reduction Output

Figure 5: Vergelijking tussen (a) CPU, (b) GPU Atomic en (c) GPU Reduction resultaten.

Hoe stijgt de tijd als functie van de input?

Theoretisch wordt verwacht dat de uitvoeringstijd van de CPU-benadering lineair toeneemt ($O(N)$), aangezien elk element opeenvolgend verwerkt moet worden. Bij de GPU-kernels liggen de verwachtingen anders door de parallelle architectuur. Voor de atomic benadering wordt een toename in tijd verwacht naarmate de input groeit, niet alleen door de hoeveelheid data, maar vooral door de toenemende threads die vaker op elkaar moeten wachten om seriële toegang te krijgen tot de gedeelde geheugenlocatie. De reductie-methode zou theoretisch de meest efficiënte schaalbaarheid moeten tonen. Om de berekeningen nog sneller te maken kan er gebruikgemaakt worden van shared memory in plaats van globale memory.

Referentie: Werking atomic operation voordelen / nadelen
Referentie: Werking reduction principe + mogelijke verbeteringen

Bij de metingen (figuur 6) zien we dat de CPU bewerkingen een lineair verloop hebben, zoals verwacht werd. Voor de atomic en reduction operatie zien we dat het voor kleinere array groottes meer tijd nodig heeft om de maximale waarde te vinden. Voor kleinere arrays is de CPU dus het efficiënst. Vanaf een array grootte van 50 000 elementen is de GPU sneller dan de CPU, de tijd die

de atomic operation en reduction nodig hebben is ongeveer gelijk.

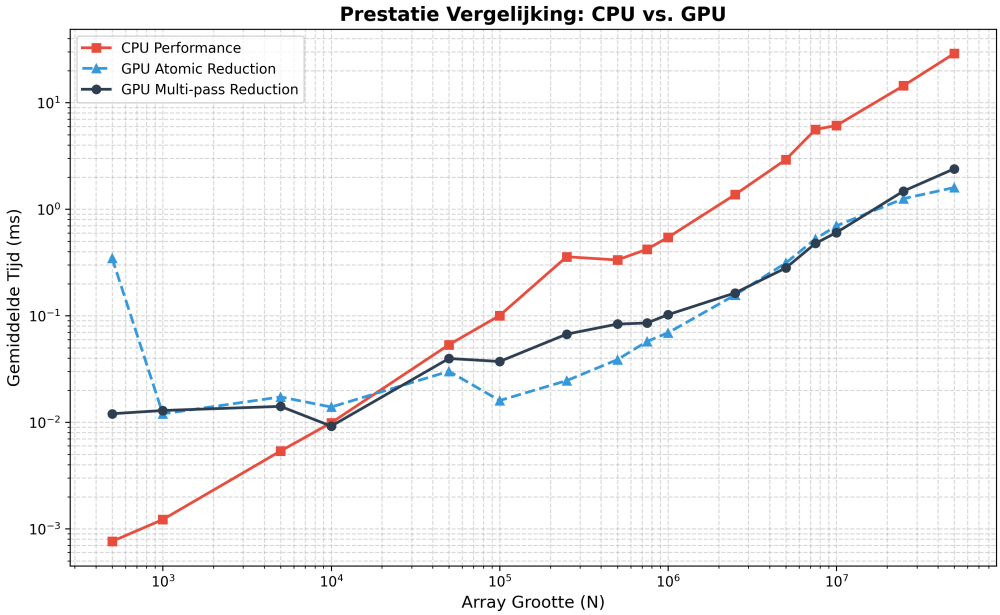


Figure 6: Vergelijking prestaties

Wat gebeurt er als de input groter is dan 2048 elementen?

Wanneer de input groter is dan 2048 elementen, overschrijdt dit de maximale blockgrootte van de meeste moderne GPU's (vaak beperkt tot 1024 threads per block). Om een correcte berekening te garanderen, moet de data over meerdere blocks worden verdeeld. Omdat threads alleen binnen hun eigen block kunnen synchroniseren via `__syncthreads()`, is er extra globale synchronisatie nodig. Dit kan door de kernel meerdere keren aan te roepen (waarbij de output van de eerste fase de input van de tweede fase wordt). Zonder deze extra stappen kunnen race conditions ontstaan tussen blocks, aangezien er geen garantie is over de volgorde waarin blocks worden uitgevoerd. In de volgende referentie staat wat er juist gebeurt en wat de problemen zijn en hoe deze operaties nog eficiënter geïmplementeerd kunnen worden.

Referentie: Uitleg reduction

Hoe zorg je dat alle elementen verwerkt worden bij minder threads dan data?

Wanneer het aantal threads kleiner is dan het totale aantal data-elementen,

dan kan je gebruikmaken van een *grid-stride loop*. In plaats van dat elke thread slechts één index verwerkt ($index = blockIdx.x * blockDim.x + threadIdx.x$), gebruikt de thread een loop die telkens springt met de totale grootte van het grid ($stride = gridDim.x * blockDim.x$). De loop ziet er als volgt uit: `for (int i = index; i < n; i += stride)`. Hierdoor verwerkt elke thread meerdere elementen sequentieel totdat de volledige array is doorlopen. Deze aanpak garandeert niet alleen dat alle elementen verwerkt worden, maar is ook zeer flexibel omdat de kernel onafhankelijk wordt van de specifieke grid-configuratie.

Referentie: Uitleg grid-stride loop

How would you tackle a 2D array?

Een 2D array is een array waar elk element een apparte array is.Een manier om het maximum te vinden van deze array zou bestaan uit het maximum vinden van elke geneste array en dan deze maxima te vergelijken met elkaar. Efficienter zou zijn om de 2d array te beschouwen als 1 grote array en de elementen te laten volgen op volgorde van 1st alle elementen van array 1 en dan alle elementen van array 2...
Nu overloop je elk element van de 1ste array en ken je die het 1st volgend threadId (+block * blocksize) toe, als deze is doorlopen doorloop je de 2de array en ken je deze de volgende id's toe. Eens dit gebeurd is kan je reductie of atomicMax gebruiken om de maximale waarde te vinden.
Het kolom ID is dan de gehele deling het id van de thread over het aantal kolommen en de rij ID is dan het verschil tussen de uitkomst van die gehele deling maal de rij-groottes en de thread-id zelf.
Deze manier vergt wel wat voorbereidingswerk maar met door elke array appart te gaan en daarna nog eens alle elementen vergelijken kan dit sneller zijn denk ik.
Omdat je het voorbereidend werk in parralel op de GPU zou kunnen doen, elke thread heeft een ander stukje data nodig, dit in tegenstelling tot de 1ste manier waar je alles gesynchroniseerd zou moeten doen.

4 Referenties

https://supercomputingblog.com/cuda/cuda-tutorial-4-atomic-operations/

5 Feedback vorig labo

Timing van operaties niet in tabel in een grafiek zetten x-as array grootte en y-as tijd

6 Appendix + simulatiewaarden in tabel

6.1 Code voor nested for loop on the CPU

```
#include <iostream>
#include <chrono>
#include <vector>
#include <limits.h>

void testCPU(int n) {
    int* h_array = (int*)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        h_array[i] = rand() % 1000000;
    }

    double totale_tijd_s = 0;
    int max_val = INT_MIN;

    for (int run = 0; run < 10; run++) {
        const auto start = std::chrono::steady_clock::now();

        max_val = INT_MIN;
        for (int i = 0; i < n; i++) {
            if (h_array[i] > max_val) {
                max_val = h_array[i];
            }
        }

        const auto end = std::chrono::steady_clock::now();
        const std::chrono::duration<double> elapsed_seconds{
            end - start};

        totale_tijd_s += elapsed_seconds.count();
    }

    double gem_tijd_s = totale_tijd_s / 10.0;
    double gem_tijd_ms = gem_tijd_s * 1000.0;

```

```
    printf("CPU | N: %10d | Max: %7d | Gem. Tijd: %f ms (%f s)\n",
           n, max_val, gem_tijd_ms, gem_tijd_s);

    free(h_array);
}

int main() {
    int test_sizes[] = {100, 1000, 10000, 100000, 1000000, 10000000};

    for (int n : test_sizes) {
        testCPU(n);
    }

    return 0;
}
```

Table 1: CPU-prestaties voor verschillende arraygroottes (N)

	Array Grootte (N)	Gem. Tijd (ms)
1	500	0.000 762
2	1000	0.001 222
3	5000	0.005 372
4	10 000	0.009 899
5	50 000	0.053 452
6	100 000	0.100 456
7	250 000	0.357 938
8	500 000	0.333 686
9	750 000	0.421 012
10	1 000 000	0.544 367
11	2 500 000	1.370 939
12	5 000 000	2.921 997
13	7 500 000	5.602 125
14	10 000 000	6.106 733
15	25 000 000	14.416 853
16	50 000 000	28.945 221

6.2 Code voor GPU kernel employing atomic operation

```
#include <stdio.h>
#include <cuda_runtime.h>
#include <stdlib.h>
#include <time.h>
#include <limits.h>
```

```
6
7   __global__ void findMaxAtomic(int* d_array, int* d_max, int
        n) {
8       int idx = blockIdx.x * blockDim.x + threadIdx.x;
9       if (idx < n) atomicMax(d_max, d_array[idx]);
10  }
11
12  void testAtomicMax(int n) {
13      int *h_array = (int*)malloc(n * sizeof(int));
14      int h_max_init = INT_MIN;
15      int h_result;
16
17      for (int i = 0; i < n; i++) h_array[i] = rand() %
            1000000;
18
19      int *d_array, *d_max;
20      cudaMalloc(&d_array, n * sizeof(int));
21      cudaMalloc(&d_max, sizeof(int));
22
23      int threadsPerBlock = 512;
24      int blocksPerGrid = (n + threadsPerBlock - 1) /
            threadsPerBlock;
25      const int herhalingen = 10;
26      float totale_tijd = 0;
27
28      cudaMemcpy(d_array, h_array, n * sizeof(int),
            cudaMemcpyHostToDevice);
29
30      for (int i = 0; i < herhalingen; i++) {
31          cudaMemcpy(d_max, &h_max_init, sizeof(int),
                cudaMemcpyHostToDevice);
32
33          cudaEvent_t start, stop;
34          cudaEventCreate(&start);
35          cudaEventCreate(&stop);
36
37          cudaEventRecord(start);
38          findMaxAtomic<<<blocksPerGrid, threadsPerBlock>>>(
                d_array, d_max, n);
39          cudaEventRecord(stop);
40          cudaEventSynchronize(stop);
41
42          float ms = 0;
43          cudaEventElapsedTime(&ms, start, stop);
44          totale_tijd += ms;
45
46          if (i == herhalingen - 1) {
47              cudaMemcpy(&h_result, d_max, sizeof(int),
                    cudaMemcpyDeviceToHost);
48          }
```

```
49
50          cudaEventDestroy(start);
51          cudaEventDestroy(stop);
52      }
53
54      printf("N: %10d | Max: %7d | Tijd: %.4f ms\n", n,
            h_result, totale_tijd / herhalingen);
55
56      cudaFree(d_array);
57      cudaFree(d_max);
58      free(h_array);
59  }
60
61  int main() {
62      srand(time(NULL));
63      int test_sizes[] = {
64          500, 1000, 5000, 10000, 50000, 100000, 250000,
                500000, 750000,
65          1000000, 2500000, 5000000, 7500000, 10000000,
                25000000, 50000000
66      };
67      int num_tests = sizeof(test_sizes) / sizeof(int);
68
69
70      printf("Start test:\n");
71      for (int i = 0; i < num_tests; i++) {
72          testAtomicMax(test_sizes[i]);
73      }
74      return 0;
75  }
```

Table 2: GPU-prestaties (CUDA Atomic Max) voor verschillende arraygroottes (N)

	Array Grootte (N)	Gem. Tijd (ms)
1	500	0.347 100
2	1 000	0.012 000
3	5 000	0.017 300
4	10 000	0.013 900
5	50 000	0.030 100
6	100 000	0.016 000
7	250 000	0.024 600
8	500 000	0.038 700
9	750 000	0.057 400
10	1 000 000	0.069 200
11	2 500 000	0.156 600
12	5 000 000	0.313 000
13	7 500 000	0.526 000
14	10 000 000	0.698 900
15	25 000 000	1.256 200
16	50 000 000	1.596 600

6.3 Code voor GPU kernel using reduction

```
#include <stdio.h>
#include <cuda_runtime.h>
#include <stdlib.h>
#include <time.h>
#include <limits.h>

__global__ void findMaxReductionKernel(int* d_in, int* d_out
    , int n) {
    int tid = threadIdx.x;
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    // Elke pass halveert het aantal actieve threads in een
        block
    for (unsigned int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s && (i + s) < n) {
            if (d_in[i + s] > d_in[i]) {
                d_in[i] = d_in[i + s];
            }
        }
        __syncthreads();
    }

    // De winnaar van dit block schrijft naar de output
        array
```

```
22      if (tid == 0) {
23          d_out[blockIdx.x] = d_in[i];
24      }
25  }
26
27  void testReduction(int n) {
28      int *h_array = (int*)malloc(n * sizeof(int));
29      for (int i = 0; i < n; i++) h_array[i] = rand() %
            1000000;
30
31      int *d_data_A, *d_data_B;
32      cudaMalloc(&d_data_A, n * sizeof(int));
33      cudaMalloc(&d_data_B, n * sizeof(int));
34
35      const int herhalingen = 10;
36      float totale_tijd = 0;
37      int h_result;
38
39      for (int r = 0; r < herhalingen; r++) {
40          cudaMemcpy(d_data_A, h_array, n * sizeof(int),
                cudaMemcpyHostToDevice);
41
42          cudaEvent_t start, stop;
43          cudaEventCreate(&start);
44          cudaEventCreate(&stop);
45          cudaEventRecord(start);
46
47          int current_n = n;
48          int threadsPerBlock = 512;
49          int* d_ptr_in = d_data_A;
50          int* d_ptr_out = d_data_B;
51
52          while (current_n > 1) {
53              int blocksPerGrid = (current_n + threadsPerBlock
                    - 1) / threadsPerBlock;
54              findMaxReductionKernel<<<blocksPerGrid,
                    threadsPerBlock>>>(d_ptr_in, d_ptr_out,
                    current_n);
55
56              // Wissel pointers voor de volgende pass
57              int* temp = d_ptr_in;
58              d_ptr_in = d_ptr_out;
59              d_ptr_out = temp;
60
61              current_n = blocksPerGrid;
62          }
63
64          cudaEventRecord(stop);
65          cudaEventSynchronize(stop);
66          float ms = 0;
```

```
67          cudaEventElapsedTime(&ms, start, stop);
68          totale_tijd += ms;
69
70          if (r == herhalingen - 1) {
71              cudaMemcpy(&h_result, d_ptr_in, sizeof(int),
                    cudaMemcpyDeviceToHost);
72          }
73
74          cudaEventDestroy(start);
75          cudaEventDestroy(stop);
76      }
77
78      printf("N: %10d | Max: %7d | Tijd: %.4f ms\n", n,
            h_result, totale_tijd / herhalingen);
79
80      cudaFree(d_data_A);
81      cudaFree(d_data_B);
82      free(h_array);
83  }
84
85  int main() {
86      srand(1337);
87
88      int test_sizes[] = {
89          500, 1000, 5000, 10000, 50000, 100000, 250000,
                500000, 750000,
90          1000000, 2500000, 5000000, 7500000, 10000000,
                25000000, 50000000
91      };
92      int num_tests = sizeof(test_sizes) / sizeof(int);
93
94      printf("Start test (Multi-pass Reduction):\n");
95      for (int i = 0; i < num_tests; i++) {
96          testReduction(test_sizes[i]);
97      }
98
99      return 0;
100 }
```

Table 3: GPU-prestaties (CUDA Multi-pass Reduction) voor verschillende arraygroottes (N)

	Array Grootte (N)	Gem. Tijd (ms)
1	500	0.012 030
2	1 000	0.012 900
3	5 000	0.014 100
4	10 000	0.009 200
5	50 000	0.039 700
6	100 000	0.037 200
7	250 000	0.067 200
8	500 000	0.083 500
9	750 000	0.085 500
10	1 000 000	0.102 400
11	2 500 000	0.163 700
12	5 000 000	0.280 800
13	7 500 000	0.477 800
14	10 000 000	0.604 000
15	25 000 000	1.479 200
16	50 000 000	2.395 100